

```

4      t += counter++;      /* +1 */
5      t += counter++;      /* +2 */
6      t += counter++;      /* +3 */
7      return t;
8  }
```

这样的转换既减少了函数调用的开销，也允许对展开的代码做进一步优化。例如，编译器可以统一 `func1` 中对全局变量 `counter` 的更新，产生这个函数的一个优化版本：

```

1  /* Optimization of inlined code */
2  long funclopt() {
3      long t = 4 * counter + 6;
4      counter += 4;
5      return t;
6  }
```

对于这个特定的函数 `f` 的定义，上述代码忠实地重现了 `func1` 的行为。

GCC 的最近版本会尝试进行这种形式的优化，要么是被用命令行选项 “`-finline`” 指示时，要么是使用优化等级 `-O1` 或者更高的等级时。遗憾的是，GCC 只尝试在单个文件中定义的函数的内联。这就意味着它将无法应用于常见的情况，即一组库函数在一个文件中被定义，却被其他文件内的函数所调用。

在某些情况下，最好能阻止编译器执行内联替换。一种情况是用符号调试器来评估代码，比如 GDB，如 3.10.2 节描述的一样。如果一个函数调用已经用内联替换优化过了，那么任何对这个调用进行追踪或设置断点的尝试都会失败。还有一种情况是用代码剖析的方式来评估程序性能，如 5.14.1 节讨论的一样。用内联替换消除的函数调用是无法被正确剖析的。

在各种编译器中，就优化能力来说，GCC 被认为是胜任的，但是并不是特别突出。它完成基本的优化，但是它不会对程序进行更加“有进取心的”编译器所做的那种激进变换。因此，使用 GCC 的程序员必须花费更多的精力，以一种简化编译器生成高效代码的任务的方式来编写程序。

5.2 表示程序性能

我们引入度量标准每元素的周期数 (Cycles Per Element, CPE)，作为一种表示程序性能并指导我们改进代码的方法。CPE 这种度量标准帮助我们在更细节的级别上理解迭代程序的循环性能。这样的度量标准对执行重复计算的程序来说是很适当的，例如处理图像中的像素，或是计算矩阵乘积中的元素。

处理器活动的顺序是由时钟控制的，时钟提供了某个频率的规律信号，通常用千兆赫兹 (GHz)，即十亿周期每秒来表示。例如，当表明一个系统有 “4GHz” 处理器，这表示处理器时钟运行频率为每秒 4×10^9 个周期。每个时钟周期的时间是时钟频率的倒数。通常是以纳秒 (nanosecond, 1 纳秒等于 10^{-9} 秒) 或皮秒 (picosecond, 1 皮秒等于 10^{-12} 秒) 为单位的。例如，一个 4GHz 的时钟其周期为 0.25 纳秒，或者 250 皮秒。从程序员的角度来看，用时钟周期来表示度量标准要比用纳秒或皮秒来表示有帮助得多。用时钟周期来表示，度量值表示的是执行了多少条指令，而不是时钟运行得有多快。

许多过程含有在一组元素上迭代的循环。例如，图 5-1 中的函数 `psum1` 和 `psum2` 计算的都是一个长度为 n 的向量的前置和 (prefix sum)。对于向量 $\vec{a} = \langle a_0, a_1, \dots, a_{n-1} \rangle$ ，前置和 $\vec{p} = \langle p_0, p_1, \dots, p_{n-1} \rangle$ 定义为

$$p_0 = a_0$$

$$p_i = p_{i-1} + a_i, \quad 1 \leq i < n \quad (5.1)$$

```

1  /* Compute prefix sum of vector a */
2  void psum1(float a[], float p[], long n)
3  {
4      long i;
5      p[0] = a[0];
6      for (i = 1; i < n; i++)
7          p[i] = p[i-1] + a[i];
8  }
9
10 void psum2(float a[], float p[], long n)
11 {
12     long i;
13     p[0] = a[0];
14     for (i = 1; i < n-1; i+=2) {
15         float mid_val = p[i-1] + a[i];
16         p[i] = mid_val;
17         p[i+1] = mid_val + a[i+1];
18     }
19     /* For even n, finish remaining element */
20     if (i < n)
21         p[i] = p[i-1] + a[i];
22 }

```

图 5-1 前置和函数。这些函数提供了我们如何表示程序性能的示例

函数 psum1 每次迭代计算结果向量的一个元素。第二个函数使用循环展开(loop unrolling)的技术, 每次迭代计算两个元素。本章后面我们会探讨循环展开的好处。(关于分析和优化前置和计算的内容请参见练习题 5.11、5.12 和家庭作业 5.19。)

这样一个过程所需要的时间可以用一个常数加上一个与被处理元素个数成正比的因子来描述。例如, 图 5-2 是这两个函数需要的周期数关于 n 的取值范围图。使用最小二乘拟

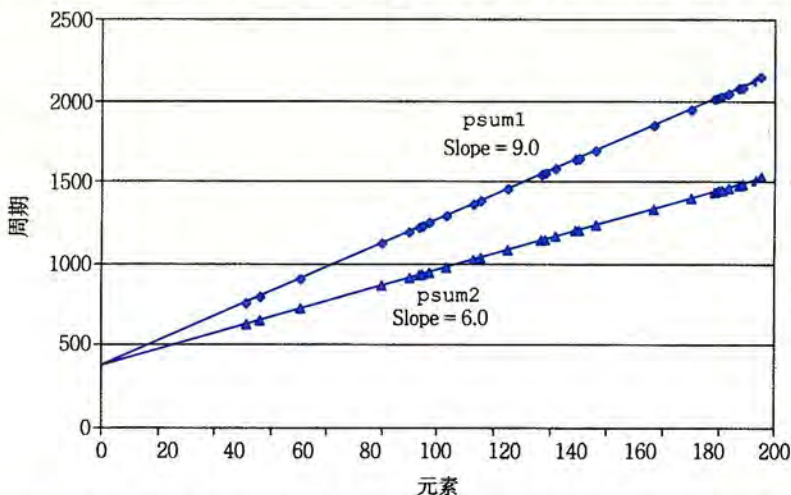


图 5-2 前置和函数的性能。两条线的斜率表明每元素的周期数(CPE)的值

合(least squares fit), 我们发现, psum1 和 psum2 的运行时间(用时钟周期为单位)分别近似于等式 $368 + 9.0n$ 和 $368 + 6.0n$ 。这两个等式表明对代码计时和初始化过程、准备循环以及完成过程的开销为 368 个周期加上每个元素 6.0 或 9.0 周期的线性因子。对于较大的 n 的值(比如说大于 200), 运行时间就会主要由线性因子来决定。这些项中的系数称为每元素的周期数(简称 CPE)的有效值。注意, 我们更愿意用每个元素的周期数而不是每次循环的周期数来度量, 这是因为像循环展开这样的技术使得我们能够用较少的循环完成计算, 而我们最终关心的是, 对于给定的向量长度, 程序运行的速度如何。我们将精力集中在减小计算的 CPE 上。根据这种度量标准, psum2 的 CPE 为 6.0, 优于 CPE 为 9.0 的 psum1。

旁注 什么是最小二乘拟合

对于一个数据点 $(x_1, y_1), \dots, (x_n, y_n)$ 的集合, 我们常常试图画一条线, 它能最接近于这些数据代表的 $X-Y$ 趋势。使用最小二乘拟合, 寻找一条形如 $y = mx + b$ 的线, 使得下面这个误差度量最小:

$$E(m, b) = \sum_{i=1, n} (mx_i + b - y_i)^2$$

将 $E(m, b)$ 分别对 m 和 b 求导, 把两个导数函数设置为 0, 进行推导就能得出计算 m 和 b 的算法。

练习题 5.2 在本章后面, 我们会从一个函数开始, 生成许多不同的变种, 这些变种保持函数的行为, 又具有不同的性能特性。对于其中三个变种, 我们发现运行时间(以时钟周期为单位)可以用下面的函数近似地估计:

版本 1: $60 + 35n$

版本 2: $136 + 4n$

版本 3: $157 + 1.25n$

每个版本在 n 取什么值时是三个版本中最快的? 记住, n 总是整数。

5.3 程序示例

为了说明一个抽象的程序是如何被系统地转换成更有效的代码的, 我们将使用一个基于图 5-3 所示向量数据结构的运行示例。向量由两个内存块表示: 头部和数据数组。头部是一个声明如下的结构:

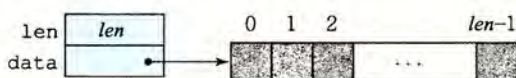


图 5-3 向量的抽象数据类型。向量由头信息加上指定长度的数组来表示

code/opt/vec.h

```
1  /* Create abstract data type for vector */
2  typedef struct {
3      long len;
4      data_t *data;
5  } vec_rec, *vec_ptr;
```

code/opt/vec.h

这个声明用 `data_t` 来表示基本元素的数据类型。在测试中, 我们度量代码对于整数(C 语言的 `int` 和 `long`)和浮点数(C 语言的 `float` 和 `double`)数据的性能。为此, 我们会分别为不同的类型声明编译和运行程序, 就像下面这个例子对数据类型 `long` 一样: